

Tecnológico de Monterrey

# Evidencia2:
# Actividad Integral de Estructuras de Datos Lineales

Allison Charlize Arriaza - A00844457

**Profesor: David Alonso Cantú**

18 de octubre, 2026

Av. Eugenio Garza
Sada 2501 Sur, Tecnológico
64700 Monterrey, N.L.

**Programación de estructuras de datos y algoritmos fundamentales**

**(Grupo 607)**

## 1. Evidencia de ejecución de programa

**Figura 1**

*Opciones de menú*

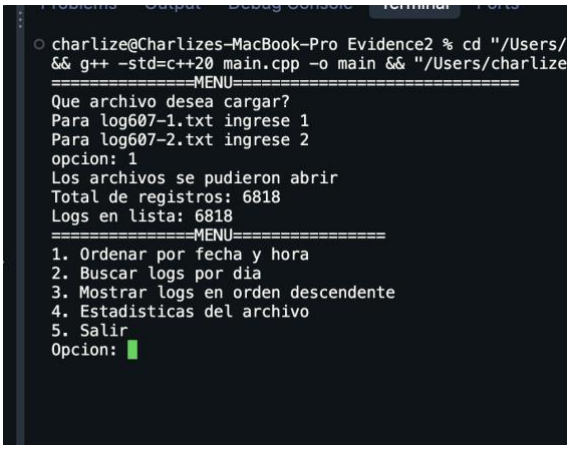



**Figura 2**

*Ordenamiento y búsqueda por fecha*

```
==========MENU==========
Que archivo desea cargar?
Para log607-1.txt ingrese 1
Para log607-2.txt ingrese 2
opcion: 1
Los archivos se pudieron abrir
Total de registros: 6818
Logs en lista: 6818
==========MENU==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 1
Que crecimiento esperas en las comparaciones?
1. n log n
2. n^2
Opcion: 1
Algotitmo: Merge Sort
Archivo: log607-1.txt
Tiempo de ejecución: 85.2076 ms
Logs ordenados: 6818
Comparaciones: 78330
Profundidad maxima: 14
La prediccion coincide con Merge Sort (n log n)
Archivo output608.out creado
==========MENU==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 2
Mes a buscar: Sep
Buscar fecha: 7
Buscar Año: 2026
----------------------------------
Sep 7 2026 00:01:24 10.14.1.130  Configuration errors
Sep 7 2026 00:35:05 10.14.34.58  Anomalous User Activity
Sep 7 2026 04:21:38 10.14.48.44  Encryption Failures
Sep 7 2026 04:39:58 10.14.43.154  Network connectivity issues
Sep 7 2026 08:19:52 10.14.237.112  Weak or easily guessable passwords
Sep 7 2026 11:42:38 10.14.229.232  Data Exfiltration Attempts
Sep 7 2026 11:52:19 10.14.48.218  Data corruption
Sep 7 2026 14:39:06 10.14.212.167  Network connectivity issues
Sep 7 2026 16:48:14 10.14.119.222  Security breaches
Sep 7 2026 17:30:32 10.14.109.102  Port Scanning Activity
Sep 7 2026 23:08:26 10.14.227.119  Data Exfiltration Attempts
Sep 7 2026 23:50:40 10.14.178.169  Security breaches
----------------------------------
Registros encontrados: 12
==========MENU==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion:
```

**Figura 3**

*Ordenamiento descendente y estadística*

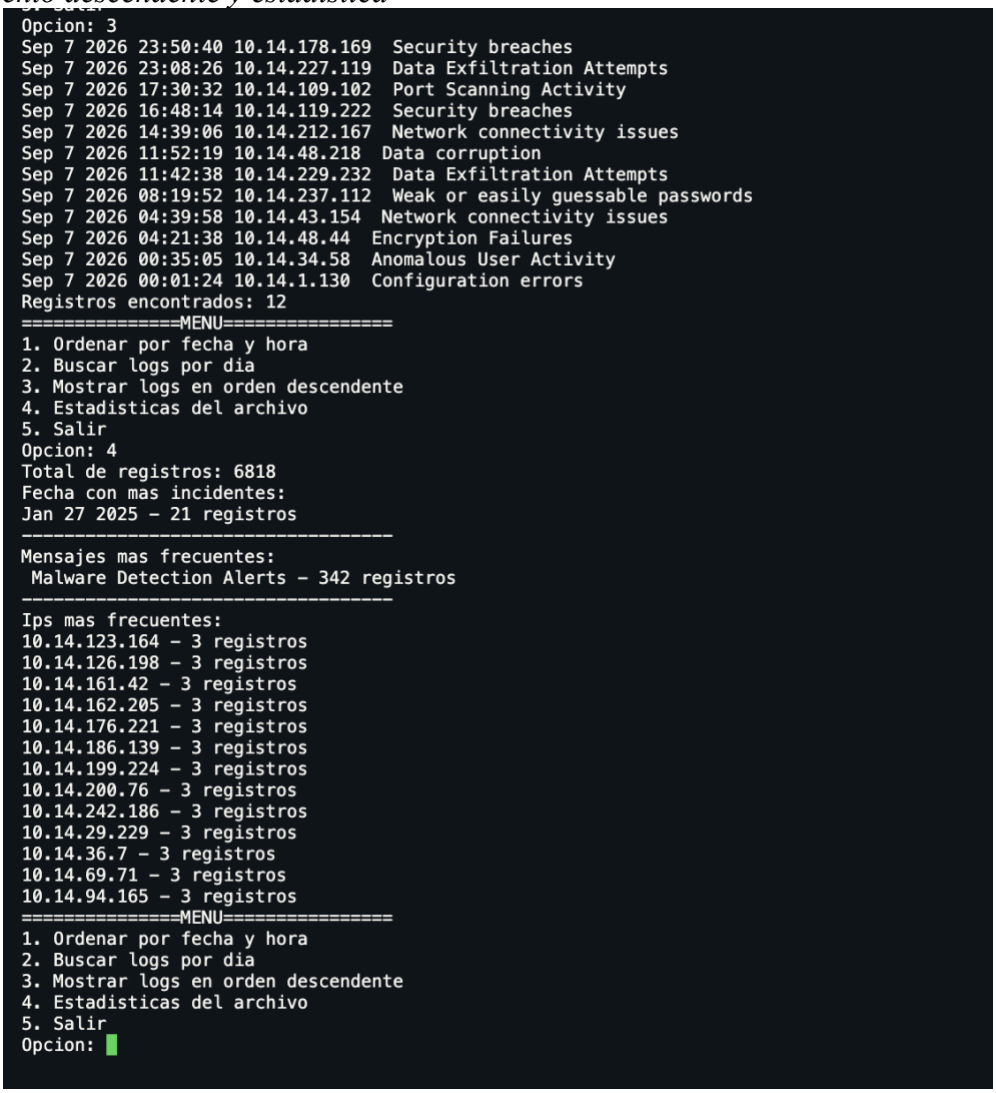
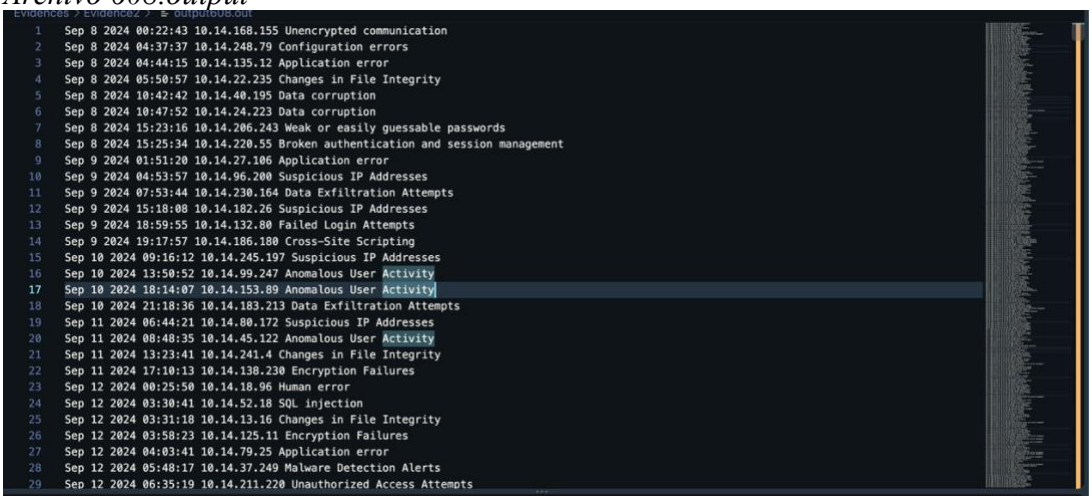


**Figura 4**

*Archivo 608.output*



```
Evidences > Evidence2 > output608.out
1   Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
2   Sep 8 2024 04:37:37 10.14.248.79 Configuration errors
3   Sep 8 2024 04:44:15 10.14.135.12 Application error
4   Sep 8 2024 05:50:57 10.14.22.235 Changes in File Integrity
5   Sep 8 2024 10:42:42 10.14.40.195 Data corruption
6   Sep 8 2024 10:47:52 10.14.24.223 Data corruption
7   Sep 8 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
8   Sep 8 2024 15:25:34 10.14.220.55 Broken authentication and session management
9   Sep 9 2024 01:51:20 10.14.27.106 Application error
10  Sep 9 2024 04:53:57 10.14.96.200 Suspicious IP Addresses
11  Sep 9 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts
12  Sep 9 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
13  Sep 9 2024 18:59:55 10.14.132.80 Failed Login Attempts
14  Sep 9 2024 19:17:57 10.14.186.180 Cross-Site Scripting
15  Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
16  Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
17  Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
18  Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts
19  Sep 11 2024 06:44:21 10.14.80.172 Suspicious IP Addresses
20  Sep 11 2024 08:48:35 10.14.45.122 Anomalous User Activity
21  Sep 11 2024 13:23:41 10.14.241.4 Changes in File Integrity
22  Sep 11 2024 17:10:13 10.14.138.230 Encryption Failures
23  Sep 12 2024 00:25:50 10.14.18.96 Human error
24  Sep 12 2024 03:30:41 10.14.52.18 SQL injection
25  Sep 12 2024 03:31:18 10.14.13.16 Changes in File Integrity
26  Sep 12 2024 03:58:23 10.14.125.11 Encryption Failures
27  Sep 12 2024 04:03:41 10.14.79.25 Application error
28  Sep 12 2024 05:48:17 10.14.37.249 Malware Detection Alerts
29  Sep 12 2024 06:35:19 10.14.211.220 Unauthorized Access Attempts
```

**Figura 5**

*Pruebas Parte A*

```
========== Linked list ==========
Lista vacia:
Tamaño: 0
Lista con datos: 10-20-30
Despues de borrar el primero: 20-30
Despues de borrar el ultimo: 20
Tamaño despues de vaciar la lista: 0
Nuevo dato: 50

======== Queue ===========
Vacia: 1
Tamaño: 3
Primero: 10
Dato: 10
Dato: 20
Dato: 30
Tamaño final: 0
Nuevo primer dato: 40

========== Stack ===========
Vacia: 1
Tamaño: 3
Arriba: 30
Dato: 30
Dato: 20
Dato: 10
Tamaño final: 0
Nuevo Dato: 40

===== Doubly Linked List =====
Tamaño inicial: 0
Imprimir Adelante: 10 20 30
Imprimir Atras: 30 20 10
Lista despues de insertar: 10 20 25 30
Lista despues de borrar el primero: 20 25 30
Lista despues de borrar el ultimo: 20 25
Tamaño al vaciar lista: 0
Nuevo dato: 50

======= Pruebas vacias =========
La pila esta vacia
La cola esta vacia
Fin de las pruebas
charlize@Charlizes-MacBook-Pro Evidence2 %
```

**Figura 6**
*Fecha de Inválida*

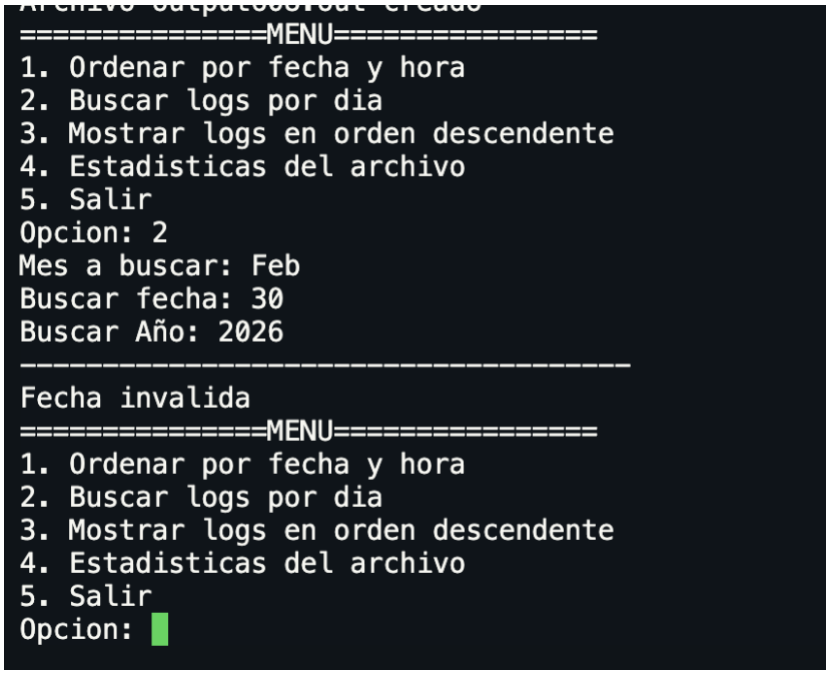



**Figura 7**
*Doble ordenamiento*

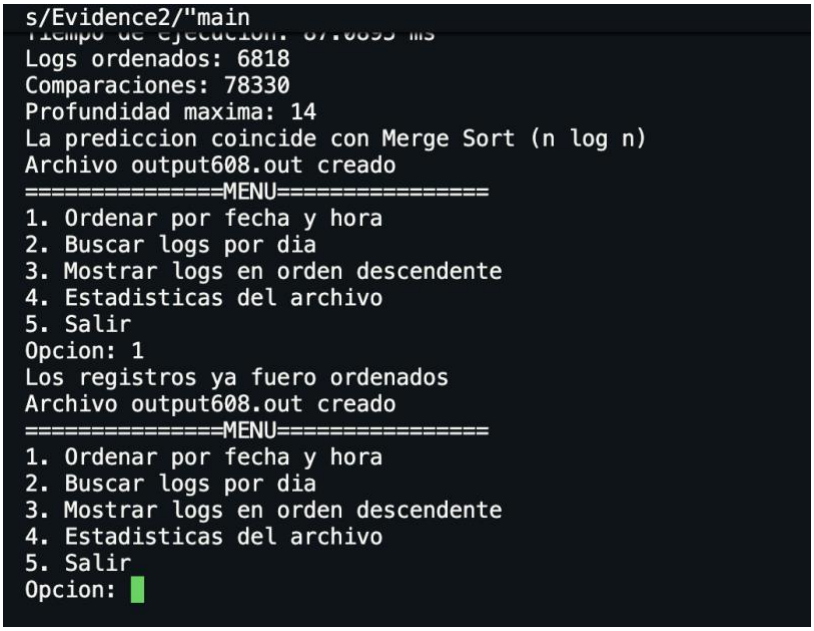


**Figura 8**

*Búsquedas antes de ordenar*




## 2. Resultado de ordenamiento

Para ordenar los registros utilicé Merge Sort y mi predicción fue O(n log n), ya que el algoritmo divide la lista en partes pequeñas y después las vuelve a unir en orden. Los dos archivos tenían 6,818 registros, pero los resultados fueron diferentes.

El primer archivo necesitó 78,330 comparaciones y tardó 82.356 ms, mientras que el segundo realizó 50,627 comparaciones y tardó 58.8995 ms. En ambos casos la profundidad máxima fue de 14. Aunque tenían la misma cantidad de datos, el segundo archivo necesitó menos comparaciones, esto muestra que el orden inicial de los registros sí influye en el trabajo que realiza el algoritmo, aunque su complejidad sigue siendo O(n log n).

**Tabla 1**
*Resultados de ordenamiento*

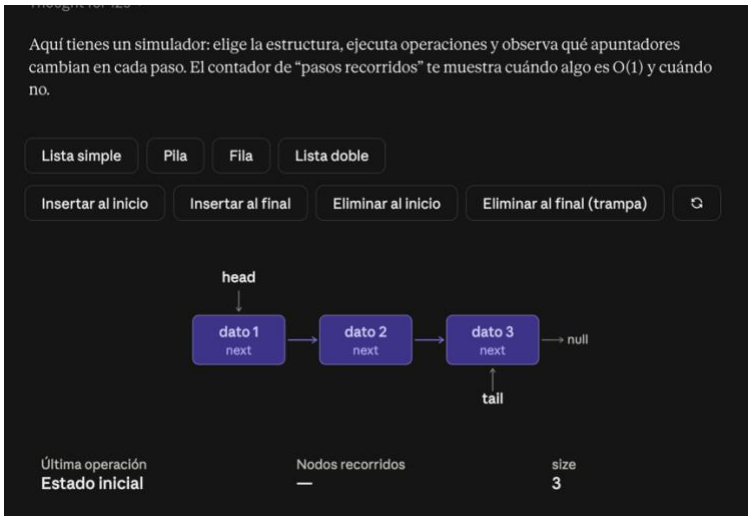
| Dato | log607-1.txt | log607-2.txt |
|---|---|---|
| Registros | 6818 | 6818 |
| Predicción | O(n log n) | O(n log n) |
| Tiempo de ejecución | 82.356 ms | 58.8995 ms |
| Comparaciones | 78,330 | 50,627 |
| Profundidad máxima | 14 | 14 |
| ¿Conservó el orden de registros iguales? | Si | Si |

## 3. Prompts utilizados

### Prompt 1

*"Este es mi proyecto, no quiero que me lo hagas, pero quiero que me ayudes a entender mediante un diagrama interactivo cómo establecer los nodos para tener un sistema O(1)."*

Este prompt lo utilicé con Claude para entender cómo debían estar conectados los nodos, me realizó un diagarama interáctivo para poderlo visualizar mejor los requisitos de la entrega y a a comprender mejor las operaciones en los extremos de las listas y cómo evitar recorrerlas innecesariamente.



**Prompt 2**

*"Ya tengo mi lista Doubly Linked list , pero no sé cómo agregar Merge Sort para ordenar los logs. Me explicas cómo hacerlo utilizando los nodos que ya tengo sin tener que cambiar toda la lista?"*

Utilicé este prompt porque ya tenía mi Doubly linked list, pero no sabía cómo implementar Merge Sort sin afectar lo que había hecho antes. La ChatGPT me ayudó a entender cómo dividir la lista y después unir los nodos en orden. Con esa explicación fui adaptando el algoritmo a mi código y probando que funcionara de manera correcta

**Prompt 3**

*"¿Cómo puedo contar cuántas veces se repiten las fechas, los mensajes y las IP de mis logs? No sé qué sería mejor usar en C++ para poder hacerlo."*

Utilicé este prompt porque no sabía cómo contar las veces que aparecía un mismo dato dentro de los registros. Al principio pensé en recorrer la lista y comparar los elementos, pero no estaba segura de si era la mejor manera de hacerlo.

ChatGpt me explicó que podía utilizar map para guardar cada dato junto con la cantidad de veces que aparecía. En esta información me basé para realizar la opción 4 del menú de mi programa, donde se cuentan las frecuencias de los días, mensajes y direcciones IP.

**Error en la sugerenacia de la IA**

Cuando estaba con el Doubly Linked List, le pregunté a ChatGPT si ya tenía todo lo que las instrucciones. Me dijo que el recorrido hacia atrás ya estaba hecho porque utilizaba prev en la función getData(). Al revisar nuevamente mi código, , me di cuenta de que eso no era exactamente lo que necesitaba, porque getData() solamente me permitía obtener un dato por posición y no mostrar toda la lista, por eso terminé agregando una función para imprimir los elementos desde tail utilizando prev. Después hice una prueba para comprobar que los datos aparecieran al revés y sí funcionó. Aunque una función utilice los apuntadores correctos, no cubría todo lo que pedía la actividad.

## 4. Reflexión

Una de las cosas que más trabajo me costó en esta evidencia fue manejar los apuntadores. Ya había trabajado con listas enlazadas anteriormente, pero todavía se me complica identificar qué conexiones tenía que modificar al insertar o eliminar un nodo. Sobre todo en la Double Linked List, porque tenía que tomar en cuenta tanto next como prev. Muchas veces tuve que regresar a revisar mis funciones y hacer pruebas para entender qué estaba pasando.

Para realizar el proyecto partí de las estructuras que ya había trabajado en clase y fui modificándolas según los requisitos de la actividad. También utilicé ChatGPT y Claude para consultar algunas dudas, principalmente sobre los nodos, Merge Sort y las estadísticas. Me ayudaron a entender cosas que no tenía claras, pero también encontré respuestas que no coincidían exactamente con los enunciados. Un ejemplo fue cuando pensé que ya tenía completo el recorrido de la Doubly Linked List porque utilizaba prev dentro de getData(), pero después tuve que agregar una función para mostrar todos los elementos hacia atrás. Sin ayuda de la IA seguramente habría tenido que buscar más información y dedicar más tiempo a las pruebas.

En la Parte B utilicé una Queue para guardar temporalmente los registros y después pasarlos a una Doubly Linked List. Queue funciona con FIFO, por lo que permite sacar los elementos en el mismo orden en que entraron. En cambio, la Double List me sirve para almacenar los registros y realizar el ordenamiento. La ventaja de esta estructura es que puedo avanzar y regresarme a un dato mientras que Queue solo me permite avanzar en la fila.

Para la opción 3 utilicé un stack, aunque realmente también habría podido recorrer la lista doble hacia atrás. Como la pila funciona con LIFO, los registros salen en el orden contrario al que fueron ingresados. Esto hace que la opción sea más sencilla de implementar utilizando push() y pop(). La desventaja es que necesito guardar temporalmente esos datos en otra estructura.

También tuve que considerar cómo realizar las búsquedas. Aunque los registros estuvieran ordenados, no me convenía utilizar búsqueda binaria directamente sobre una linked list porque no puedo acceder inmediatamente al elemento de en medio como lo haría con un arreglo.En una linked list no puedo acceder inmediatamente a cualquier posición, porque

necesito avanzar por los nodos hasta encontrar el dato, incluso llegar al elemento de en medio tendría que recorrer parte de la lista. Una alternativa sería crear un arreglo con apuntadores a los nodos para poder acceder a ellos por posición, pero eso necesitaría memoria adicional. Por eso, en mi programa utilicé un recorrido que se detiene cuando la fecha ya es mayor de la que estoy buscando.

Para el ordenamiento escogí Merge Sort porque podía trabajar directamente con los nodos de la Double linked list. Al principio no sabía cómo implementarlo sin afectar las conexiones que ya tenía. Lo que terminé haciendo fue dividir la lista en partes más pequeñas y después unirlas para comparra fechas y horas de los registros.

Al probar el programa, los dos archivos tenían 6,818 registros, pero obtuve resultados diferentes. Con log607-1.txt se realizaron 78,330 comparaciones en 82.356 ms, mientras que con log607-2.txt fueron 50,627 comparaciones en 58.8995 ms. Me llamó la atención la diferencia porque ambos tenían la misma cantidad de registros. Esto se debe al ordenamiento de los datos, aunque Merge Sort mantiene una complejidad O(n log n).

La profundidad máxima fue de 14 en los dos archivos debido a que son recursivas. Cada llamada queda pendiente en el stack hasta que termina de procesar las partes correspondientes. Como se va dividiendo aproximadamente a la mitad, la profundidad no crece de la misma manera que la cantidad de registros. No utilicé Quick Sort, así que no tuve que trabajar con un pivote.

Considero que la parte más difícil no fue conseguir que el programa mostrara resultados, sino asegurarme de que las estructuras siguieran funcionando después de cada cambio. Todavía hay algunas cosas que podría mejorar, como las validaciones de los archivos y ciertos recorridos, pero ahora tengo más claro cómo se relacionan las estructuras. También me quedó claro que debo comprobar lo que me explica una IA, porque una respuesta puede parecer correcta y aun así no cubrir exactamente lo que necesito.